

Pseudo++: An Open-Source Toolchain for Romanian Baccalaureate Pseudocode

Bujor Mihai Alexandru

Faculty of Mathematics and Computer Science
Babeş-Bolyai University, Cluj-Napoca
bujormihaialexandru@gmail.com

Rares Florin Boian

Faculty of Mathematics and Computer Science
Babeş-Bolyai University, Cluj-Napoca

Abstract—The Romanian national baccalaureate examination in Computer Science requires candidates to read, write, and reason about programs expressed in a pseudocode dialect that uses Romanian keywords, Unicode mathematical symbols, and a box-drawing block structure. Despite being central to the secondary-school CS curriculum, this dialect has never had a standard execution environment: students must trace programs by hand, with no automated way to verify correctness.

This paper presents Pseudo++, an open-source toolchain that fills this gap. The toolchain is built around a single C library compiled to both a native binary and a WebAssembly module, ensuring identical behaviour in the command-line interface and the browser-based web editor. It comprises nine components: a Tree-sitter grammar formally capturing the dialect’s syntax; a two-pass formatter normalising Unicode and ASCII notation; a tree-walking interpreter with an explicit execution stack; a step-through debugger with live variable and condition inspection; a two-pass transpiler generating C, C++, or Pascal; a loop-equivalence module converting between all four loop constructs; a static analyzer reporting twelve diagnostic categories without executing the program; a browser-based editor built on Monaco, running entirely offline; and a CLI for scripting.

The toolchain is evaluated on 53 official Romanian baccalaureate problems spanning six examination years (2020–2025), achieving a 100 % pass rate. The transpiler produces compilable, warning-free C for all 32 verified problems, and all 9 directed loop-equivalence conversions preserve observable output exactly. Pseudo++ is the only tool that combines execution, debugging, transpilation, loop-equivalence transformation, and static analysis for this dialect in a single browser-based, installation-free environment.

Index Terms—pseudocode interpreter, educational programming tools, WebAssembly, transpiler, Romanian baccalaureate, Tree-sitter

I. INTRODUCTION

The Romanian national baccalaureate examination in Computer Science requires candidates to read, write, and reason about programs in a pseudocode dialect established by convention through high-school CS textbooks and examination papers. The dialect uses Romanian keywords (*citeşte*, *scrie*, *dacă*, *cât timp*, *pentru*, *repetă*), Unicode mathematical symbols for assignment and comparison ($\leftarrow$, $\leq$, $\geq$, $\neq$), and box-drawing characters ($\lfloor$, $\blacksquare$) for visual block structure.

Despite being one of the core notations in Romanian secondary CS education, this dialect has no standard execution environment. Students must trace programs by hand. Manual tracing develops intuition but is error-prone for longer programs

and provides no automated feedback. The dialect cannot be fed to any general-purpose language tool: its Unicode symbols and Romanian keywords are not valid in C, C++, or Pascal. General pseudocode interpreters designed for other language curricula do not recognise the Romanian syntax, and the only existing browser-based tool for Romanian pseudocode (`pseudocod.ro`) diverges from the baccalaureate standard in significant ways [1].

This paper presents **Pseudo++**, a complete open-source toolchain that addresses this gap. Section II describes the pseudocode dialect and its parsing challenges. Section III surveys related work. Section IV presents the system architecture. Section V details the nine core components. Section VI reports the evaluation results.

II. THE BACCALAUREATE PSEUDOCODE DIALECT

A. Notation and Language Features

The dialect combines Romanian keywords, Unicode mathematical symbols, and a visual indentation system based on box-drawing characters. Listing 1 shows a representative fragment from the 2020 baccalaureate training set: the program reads a natural number n then, for each of its digits, reads an additional value and extracts a specific digit from it using a nested for loop and floor division, accumulating the result in m .

```
citeşte n
p←1; m←0; k←0
cât timp n≠0 execută
    citeşte x
    pentru i←1, k execută
        x←[x/10]
    ■
    dacă x≠0 atunci c←x%10
    altfel c←n%10
    ■
    m←c*p+m
    n←[n/10]
    p←p*10; k←k+1
    ■
scrie m
```

Listing 1. A 2020 baccalaureate training problem in canonical Unicode notation.

The language supports all four loop constructs (`cat timp/while`, `pentru/for`, `executa...cat timp/do-while`, `repete...pana cand/repeat-until`), conditional branching (`daca...atunci...altfel`), I/O (`citeste/scrie`), assignment ($\leftarrow$), the swap operator

($x \leftrightarrow y$), floor division (`[expr]`), square root ($\sqrt{}$), modulo, and single-line comments (`#`). Multi-statement lines separated by semicolons are also supported. The dialect has no user-defined functions, arrays, or types.

B. Parsing Challenges

Three properties make the dialect non-trivial to parse. First, the canonical form mixes Unicode characters with ASCII text, requiring normalisation before a standard parser can process it. Second, block structure is encoded via box-drawing characters (`|` as indentation, `■` as a block closer), not keywords — these must be translated to whitespace and the `sf` keyword. Third, the dialect has no official formal grammar: it is defined implicitly through the corpus of textbooks and past examination papers, meaning that a grammar formalisation requires a thorough study of all syntactically valid forms, including edge cases such as multi-statement lines and optional loop step arguments.

III. RELATED WORK

PSeInt [2] is the closest analogue: a desktop pseudocode interpreter and debugger for the Spanish-speaking educational community. It requires installation, does not recognise Romanian keywords or Unicode mathematical symbols, and provides no transpiler. Having been actively developed for over two decades it has accumulated a large user base across Latin America, demonstrating the appetite for dialect-specific pseudocode tooling — an appetite that no equivalent tool has yet served for Romanian.

RAPTOR [3] and **Flowgorithm** [4] take a flowchart-based approach. Flowgorithm can export to C and C++, making it the closest existing analogue to Pseudo++’s transpiler, but its input is a graphical flowchart rather than text, so it cannot process the notation that appears on Romanian exam papers.

Pseudocod.ro [1] is the only browser-based tool that targets Romanian pseudocode directly. Its accepted syntax diverges from the baccalaureate standard, requiring students to manually adapt programs. It provides basic execution only: no debugger, no transpiler, no static analysis, and no loop-equivalence feature.

Online platforms such as Replit [5] and OnlineGDB [6] require translation to a real language. Python Tutor [7] provides step-through visualisation but only for mainstream languages. Its variable-inspection panel — a live display of name, value, and type updated after every statement — is the closest precedent for Pseudo++’s debugger view, but it cannot process any form of pseudocode.

Ben-Ari [8] advocates for executable, feedback-rich environments as a means of supporting constructivist learning in CS education. Pseudo++ operationalises this principle for the specific case of the Romanian baccalaureate dialect, combining execution, debugging, and static feedback in a single tool.

Table I summarises the feature comparison. Pseudo++ is the only tool with all six capabilities; none of the surveyed tools natively executes the Romanian baccalaureate dialect.

TABLE I
FEATURE COMPARISON OF EDUCATIONAL PROGRAMMING TOOLS. *Trans*: TRANSPILATION TO A COMPILABLE LANGUAGE. *Lint*: STATIC ANALYSIS / SEMANTIC WARNINGS. *Web*: BROWSER-BASED, NO INSTALLATION. *OSS*: OPEN SOURCE.

Tool	Exec	Debug	Trans	Lint	Web	OSS
Pseudocod.ro	✓	×	×	×	✓	×
PSeInt	✓	✓	×	×	×	✓
RAPTOR	✓	✓	×	×	×	×
Flowgorithm	✓	✓	✓	×	×	×
Replit	✓	partial	×	×	✓	×
Python Tutor	✓	✓	×	×	✓	✓
Pseudo++	✓	✓	✓	✓	✓	✓

IV. SYSTEM DESIGN

A. Architecture

The toolchain is structured as four layers of independent C modules (Fig. 1). The *utilities layer* (value, environment, diagnostic) provides the core runtime value type, variable store, and shared diagnostic record used by all upper layers. The *foundation layer* (parser, formatter, I/O) handles source ingestion and abstract I/O. The *capabilities layer* (runtime+debugger, transpiler, check, equivalence) implements the four processing modes. The *product layer* exposes the capabilities through two builds: a native CLI binary and a WebAssembly module compiled by Emscripten [9].

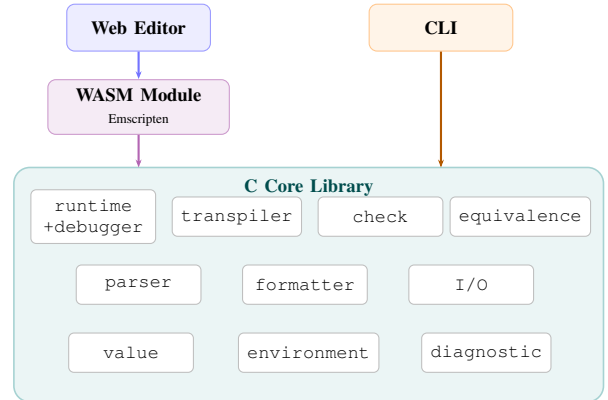


Fig. 1. Pseudo++ module dependency graph. The C core library is compiled to both a native binary (CLI) and a WebAssembly module (Web Editor), sharing all execution logic.

B. Single C Core, Two Builds

A deliberate design decision drives the architecture: the same C source is compiled twice — once to a native binary and once to a WebAssembly module. The I/O layer is the only difference between the two builds. The CLI uses `io_stdio` (standard `scanf/print`); the WASM build uses `io_buffered`, a queue-based mechanism in which JavaScript pushes input tokens and polls output tokens without blocking the browser’s

event loop. All execution, debugging, transpilation, and static analysis logic is identical between the two builds.

The WASM bridge exposes the toolchain to JavaScript via EMSCRIPTEN_KEEPAALIVE-annotated functions (selected):

```
int pseudo_init(void);
int pseudo_load(const char* source);
int pseudo_step(void);
int pseudo_run(void);
void pseudo_push_input(const char* value);
char* pseudo_pop_output(void);
char* pseudo_debug_step(void);
char* pseudo_transpile(const char* src,
                       const char* lang);
char* pseudo_check(const char* source);
```

This design guarantees that the browser and terminal interfaces produce identical output for identical input, with no server component required after the initial page load.

V. CORE COMPONENTS

A. Grammar

The language grammar is specified as a Tree-sitter [10] grammar. Tree-sitter generates a GLR parser with incremental re-parsing: edits reuse unchanged subtrees, giving $O(k \log n)$ amortised complexity for edits of size k [11]. The grammar covers all constructs described in Section II. When the parser encounters an unexpected token it inserts ERROR/MISSING sentinel nodes rather than aborting, so the resulting CST is always complete and serves as a basis for incremental diagnostics.

The interpreter and transpiler traverse the CST directly via named field accessors (condition, var, start, end, step), avoiding a separate AST lowering pass.

Expressions follow a standard precedence hierarchy encoded via Tree-sitter’s `prec.left` operator (Table II).

TABLE II
EXPRESSION PRECEDENCE LEVELS (LOWEST TO HIGHEST).

Level	Operator(s)	Node type
1	SAU / sau (or)	or_expr
2	SI / si (and)	and_expr
3	NOT / not	not_expr
4	= != < <= > >=	compare_expr
5	+ −	add_expr
6	* / %	mul_expr
7	− (x), [x], $\sqrt{x}$	neg, floor, sqrt

B. Formatter

The canonical notation uses Unicode characters ($\leq$, $\neq$, $\leftarrow$, $\blacksquare$, box-drawing bars, Romanian diacritics) that are not accepted by the parser. The formatter bridges the two representations in two passes.

Pass 1 — Symbol Substitution. A static hash map maps Unicode byte sequences to ASCII equivalents using a greedy maximum-munch strategy (e.g. $\leftarrow \rightarrow <-$, $\leq \rightarrow <=$, the box-drawing bar $| \rightarrow$ four spaces, $\blacksquare \rightarrow sf$).

Pass 2 — Structural Re-indentation. All leading whitespace is stripped and recomputed from scratch based on indent-trigger keywords (atunci, execută, repeta) and dedent-trigger tokens (sf, altfel, closing pana cand), producing a consistently-indented ASCII source.

C. Interpreter

Runtime values are represented by a heap-allocated tagged union (value_t) with three types: VALUE_INT (int64_t), VALUE_FLOAT (double), and VALUE_STRING. Division always yields a double (matching Romanian textbook convention); floor division ($[a/b]$) returns an integer. Variables are stored in a flat environment_t: an open-addressing hash table with linear probing, FNV-1a hashing [12], initial capacity 16, and a 70% load factor. Variables are implicitly initialised to integer zero.

The interpreter uses an *explicit execution stack* rather than recursion, serving two purposes: preventing stack overflow for deeply nested programs, and providing a natural point for the debugger to inspect state. Each exec_frame_t records the frame type, the corresponding CST node, and a phase counter encoding progress through the frame’s sub-steps. Frame types are: FRAME_PROGRAM, FRAME_BLOCK, FRAME_IF, FRAME_FOR, FRAME_WHILE, FRAME_DO_WHILE, FRAME_REPEAT. A compile-time dispatch table maps each type to its step function, making the execution loop a single indexed function call. The stack depth is bounded at compile time (MAX_STACK_DEPTH = 256).

Expressions are evaluated by a recursive eval_expr() function. Short-circuit evaluation is implemented for the logical connectives: or_expr returns 1 immediately if the left operand is truthy; and_expr returns 0 immediately if the left operand is falsy. Error propagation uses an optional value_error_t* out-parameter set on division-by-zero, type mismatch, or negative square root; any error transitions the runtime to EXEC_ERROR and halts execution. The EXEC_NEEDS_INPUT state suspends execution when a citește statement encounters an empty input buffer, resuming after the front end supplies the missing token.

D. Transpiler

Listing 2 shows the C output produced by the transpiler for Listing 1.

Transpilation proceeds in two passes. *Pass 1* (collect_vars) performs a depth-first CST walk and infers each variable’s type from its first assignment: a string literal $\rightarrow$ string, a floating-point literal $\rightarrow$ double, otherwise int. *Pass 2* (gen_stmt) emits target code using a lang_ops_t record that encapsulates all language-specific choices (operator syntax, declaration style, string types), enabling a single pass for all three targets.

Two language-specific concerns are handled explicitly: (1) Pascal’s and/or operators bind more tightly than comparisons, so the transpiler wraps each operand in parentheses when targeting Pascal; (2) Pascal string literals use single quotes with doubling for embedded quotes. Variable declarations are

```

#include <stdio.h>
#include <math.h>

int main() {
    int n;
    scanf("%d", &n);
    int p = 1, m = 0, k = 0;
    while (n != 0) {
        int x;
        scanf("%d", &x);
        for (int i = 1; i <= k; i++) {
            x = (int)(x / 10);
        }
        int c;
        if (x != 0) {
            c = x % 10;
        } else {
            c = n % 10;
        }
        m = c * p + m;
        n = (int)(n / 10);
        p = p * 10;
        k = k + 1;
    }
    printf("%d\n", m);
    return 0;
}

```

Listing 2. C code generated by the transpiler for the program in Listing 1.

hoisted inline at the start of each block for C, and into a global var section for Pascal.

E. Loop Equivalence Module

The loop equivalence module implements the mutual inter-convertibility of all four loop constructs, grounded in the Böhm–Jacopini theorem [13] and Dijkstra’s structured-programming principles [14]. This transformation is an explicit topic in the Romanian baccalaureate curriculum, and no existing tool provides automated support for it. Listing 3 shows a `pentru` loop converted to an equivalent `cat timp` loop.

```

# original
pentru i <- 1, n executa
    scrie i * i
sf

# while equivalent (generated)
i <- 1
cat timp i <= n executa
    scrie i * i
    i <- i + 1
sf

```

Listing 3. Loop equivalence: a `pentru` (for) loop (top) converted to a `cat timp` (while) loop (bottom).

Given a source string, a 0-indexed target line, and the desired output construct, the module: parses the source; locates the loop node via DFS; extracts header fields and body text; constructs the target form; and splices the new loop back into the source, preserving all surrounding code and indentation.

The key structural distinction is between *pre-test* loops (for, while) that may execute zero times and *post-test* loops (do-while, repeat) that always execute at least once. Two

rewrite primitives cover all nine supported directed conversions: a *guard* (wrapping the target loop in **if** *C* **then**) preserves zero-iteration semantics when converting from pre-test to post-test; *body duplication* (prepending one copy of the body before the loop) handles the reverse direction. Converting *to* a for loop is not supported, as it would require synthesising a counter variable from an arbitrary loop body.

F. Debugger

After each step, `runtime_get_variables_json()` serialises the current variable table to a JSON array of {name, value, type} objects. When the step involves a conditional or loop guard, the runtime additionally captures the condition text and its boolean result in a `condition_info_t` struct. The WASM bridge combines the variable table, current line number, execution status, and condition information into a single JSON response per `pseudo_debug_step()` call. The web front end uses this response to highlight the executing source line and update the variable inspector panel.

G. Static Analysis (check Module)

The check module traverses the CST after parsing and reports a structured set of semantic diagnostics without executing the program. Each diagnostic carries a severity (error, warning, or info), a code, a 1-indexed source line and column, and a Romanian-language message. The analyzer produces ten warning codes (W001–W010), one informational code (W011), and one error code (E001). If syntax errors are present, semantic checks are suppressed to avoid cascading false positives.

Per-node checks (DFS): W001 flags division or modulo by a literal zero; W002 flags self-assignment (`x <- x`); W004 flags empty block bodies.

Constant-expression evaluation with propagation: The analyzer embeds a constant folder covering arithmetic, comparisons, and logical connectives, including complementary identities ($A \text{ SI NOT } A \equiv \text{false}$, $A \text{ SAU NOT } A \equiv \text{true}$). A sequential propagation pass additionally tracks known variable values across statements, enabling pattern detection such as `x <- 5; daca x = 5 atunci...` W003 reports statically-constant conditions (always-true or always-false); W008 reports the unreachable code that results — dead branches in conditionals and code following infinite loops.

Multi-statement pattern checks: W005 detects the manual swap triplet (`t <- a; a <- b; b <- t`) and suggests the built-in `a <-> b` operator; W009 detects a value read from input and immediately overwritten before any use.

Usage and control-flow checks: W006 flags modification of a `pentru` loop counter inside its own body; W007 performs a two-pass whole-program analysis to identify variables assigned but never subsequently read; W010 detects shadowed loop variables in nested `pentru` loops.

Informational hints: W011 suggests simplifications for redundant boolean expressions ($x \text{ SI } x \rightarrow x$, $\text{NOT NOT } x \rightarrow x$).

The `check` subcommand prints each diagnostic as a single line (`avertisment[CODE] la linia L, coloana C: mesaj`). The `pseudo_check` WASM export serialises all diagnostics as a JSON array for the web editor’s inline gutter markers.

H. Web Editor

The web editor is a single-page application built on Monaco Editor [15] with a custom Monarch [16] tokeniser that provides syntax highlighting for keywords, operators, strings, numbers, and comments. At runtime the JavaScript application loads the compiled WASM module and communicates with it through the exported C functions, providing Run, Step Into, Continue, Transpile (with target language selection), Check, and Loop Equivalence operations. The editor requires no installation and runs entirely offline once the page is loaded, making it accessible in school lab environments where students typically lack administrator rights.

Fig. 2 shows the editor with three features active simultaneously. *Visual Mode* (toggled via the button in the toolbar) renders the underlying ASCII source as canonical Unicode notation: `<-` $\rightarrow$ $\leftarrow$, `!=` $\rightarrow$ $\neq$, `sf` $\rightarrow$ $\blacksquare$. The step-through debugger has advanced to step 9: the currently executing line is highlighted, the condition panel shows `n  $\neq$  0  $\rightarrow$  TRUE`, and the variable table lists all live bindings with their current values. The static analyser has marked line 2 with a diagnostic squiggly because `x` is assigned but never subsequently read.

I. Command-Line Interface

The `pseudo` binary exposes the same capabilities as the web editor through seven subcommands suited to batch processing and scripting: `run`, `format`, `check`, `parse`, `debug`, `transpile`, and `equivalence`. Listing 4 shows a representative session: `check` reports the unused-variable diagnostic; `run` executes the program; and `transpile` produces C code that GCC compiles to a binary producing identical output.

```
$ pseudo check demo.pseudo
avertisment[W007] la linia 2, coloana 1: Variabila '
  x' este definita dar niciodata folosita
$ pseudo run demo.pseudo <<< "123"
6
$ pseudo transpile c demo.pseudo | gcc -xc -lm - -o
  demo
$ ./demo <<< "123"
6
```

Listing 4. A CLI session: static analysis, execution, and transpilation of the same program. Diagnostic messages are in Romanian, the target language of the tool.

VI. EVALUATION

A. Testing Methodology

Testing is split across three tiers. Unit tests (`make test`) exercise individual modules in isolation using a mini-

mal `ASSERT_STR_EQ/ASSERT_INT_EQ` harness. Integration tests invoke the `pseudo` CLI binary directly with a normalised source file and compare its standard output byte-for-byte against a reference. Transpiler tests extend this pipeline by compiling the generated C with `gcc -lm` and running the resulting binary on the same input.

B. Correctness on Bacalaureate Problems

The integration test dataset consists of 53 pseudocode problems drawn from the official Romanian bacalaureate training sets published by the Ministry of Education for examination years 2020–2025 (Table III). The problems cover all four loop forms, nested conditionals, string and integer arithmetic, floor-division, modulo, and multi-variable `citește` statements.

TABLE III
BACCALAUREATE INTEGRATION TEST DATASET.

Year	Problems
2020	20
2021	12
2022	5
2023	5
2024	5
2025	6
Total	53

The interpreter correctly produces the expected output for all **53 out of 53** problems (**100 %**). No failures were observed across any of the six examination years.

C. Transpiler Verification

Transpiler correctness is verified for the C target on the 32 problems from the 2020–2021 sets. All **32 out of 32** problems (**100 %**) produce compilable C code whose output matches the interpreter reference exactly. GCC reported no compilation errors or warnings for any generated file.

D. Loop Equivalence Semantic Preservation

The equivalence module is evaluated with 9 directed conversion tests, covering all ordered pairs among the four loop constructs that change the loop type. For each test, the module transforms the source loop, the interpreter executes the result with the original input, and the output is compared against the untransformed reference. All **9 out of 9** transformations (**100 %**) preserve the program’s observable output exactly.

E. Unit Tests

The formatter module is covered by 39 unit tests verifying symbol substitution for all entries in the substitution table, greedy maximum-munch ordering, edge cases (empty input, replacements at boundaries), keyword normalisation for diacritic variants, and structural re-indentation. All 39 tests pass.

The `check` module is covered by 20 unit tests, two per diagnostic code: one positive case that triggers the warning and one negative case that confirms its absence. All 20 tests pass.

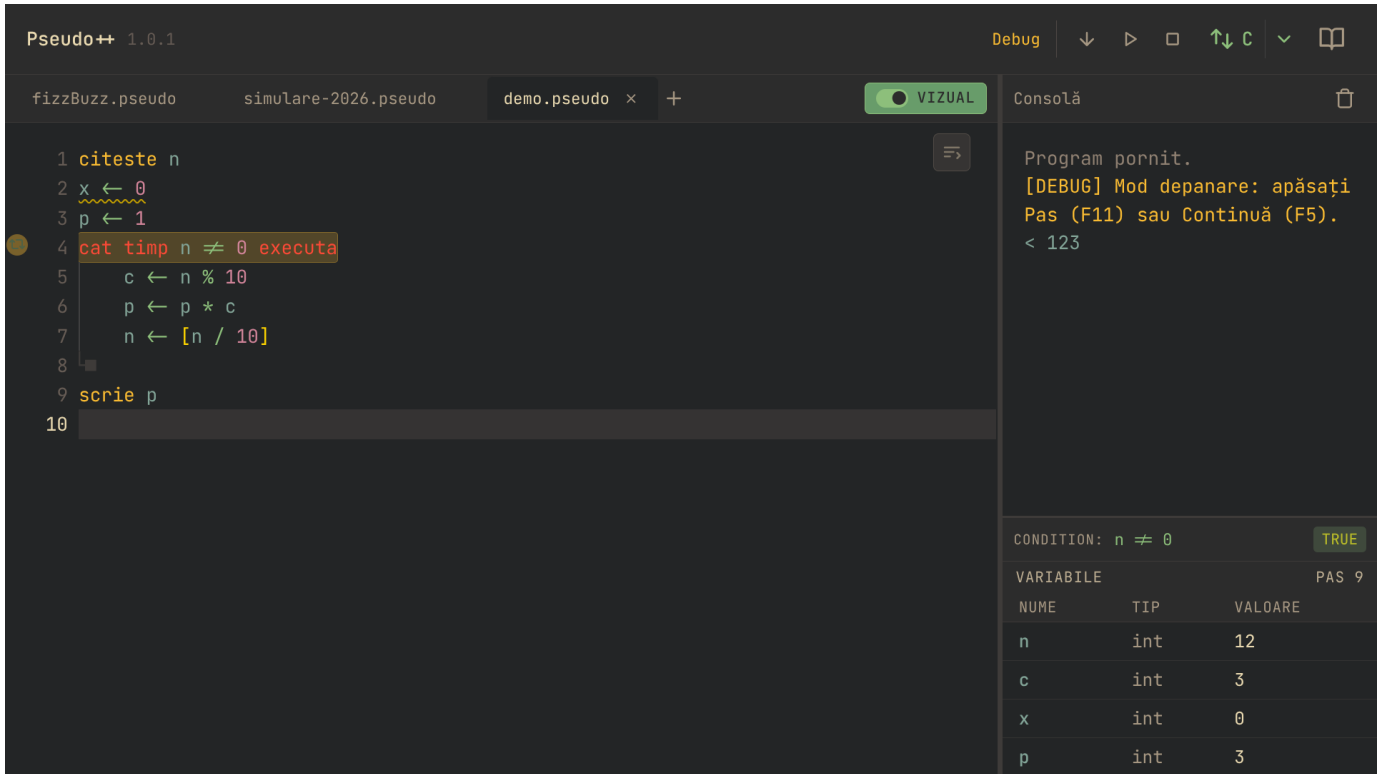


Fig. 2. The Pseudo++ web editor with Visual Mode, step-through debugger, and static analysis active simultaneously. Line 4 (cat timp $n \neq 0$ executa) is the current execution point; the condition panel confirms $n \neq 0 \rightarrow \text{TRUE}$ and the variable table shows the live state at step 9. Line 2 carries a diagnostic squiggly: the static analyser has detected that x is assigned but never read (W007). Visual Mode is active: assignment arrows, inequality symbols, and the block-close square are rendered in Unicode throughout the source pane.

F. Discussion

The 100% pass rate across six examination years (2020–2025) and 53 problems validates that the Tree-sitter grammar correctly captures the full syntax of the dialect as it appears in official Ministry of Education materials, and that the interpreter faithfully implements the semantics expected in those materials. The test set spans the full range of constructs: all four loop forms, nested conditionals, the floor-division and modulo operators, and string arithmetic.

The 100% transpiler pass rate (32/32 problems, zero GCC warnings) confirms that the two-pass variable-type inference heuristic is sufficient in practice: since the dialect lacks user-defined types, type promotion conflicts that would require a more sophisticated analysis do not arise.

A notable limitation of the evaluation is that only official baccalaureate training problems were tested. These are exclusively short, single-function programs; edge cases involving deeply nested loops, large inputs, or adversarial whitespace patterns were not covered by the test suite. CLI stress testing and in-browser informal validation confirm the absence of obvious failures in these scenarios, but a systematic treatment remains future work.

G. Limitations

Several limitations of the toolchain itself are worth stating explicitly.

Scalar variables only. The dialect processed by Pseudo++ covers only scalar types (integers, reals, strings). Arrays and user-defined functions are absent from the baccalaureate pseudocode as taught at the secondary level and are therefore outside the scope of the toolchain; programs requiring indexed data structures cannot be expressed or executed.

Forward-only debugging. The debugger supports stepping forward through execution one statement at a time but does not support stepping backwards. Inspecting an earlier program state requires restarting execution from the beginning. Snapshot-based reverse debugging [17] is identified as future work.

Heuristic re-indentation. The formatter’s structural re-indentation pass is keyword-driven and operates without a full parse. Pathological source formatting — deeply mixed Unicode and ASCII notation or unusual whitespace structure — can produce incorrect indentation, causing parse errors for programs that would otherwise be syntactically valid.

VII. CONCLUSIONS

This paper presented Pseudo++, the first open-source toolchain to provide execution, step-through debugging, transpilation to C/C++/Pascal, static analysis, and loop-equivalence transformation for the Romanian baccalaureate pseudocode dialect in a single browser-based, installation-free environment. The toolchain correctly handles all 53 official baccalaureate

problems tested across six examination years and produces correct, warning-free C code for all verified transpiler cases.

The single-C-core architecture, compiled to both a native binary and a WebAssembly module via Emscripten, ensures behavioural parity between the CLI and the web editor while eliminating any server-side infrastructure requirement.

Directions for future work include: introducing a dialect-agnostic intermediate representation to support additional Romanian pseudocode dialects (e.g. the Babeş-Bolyai University admission examination dialect) without duplicating backend logic; deepening the existing static analysis with full data-flow diagnostics and richer constant propagation; adding snapshot-based step-back debugging; and integrating the WASM module into existing Romanian educational platforms such as pbinfo.ro as a native pseudocode execution target.

Pseudo++ is available as open-source software together with the full integration test suite at <https://github.com/BujorelActimel/pseudo>.

REFERENCES

- [1] pseudocod.ro, “Pseudocod.ro: Online romanian pseudocode interpreter,” <https://www.pseudocod.ro/>, 2020.
- [2] P. Novara, “PSeInt: A pseudocode interpreter for learning programming,” <https://pseint.sourceforge.net/>, 2003.
- [3] M. C. Carlisle, T. A. Wilson, J. W. Humphries, and S. M. Hadfield, “RAPTOR: Introducing programming to non-majors with flowcharts,” in *Proceedings of the 36th ACM SIGCSE Technical Symposium on Computer Science Education*, 2005, pp. 52–56.
- [4] M. C. Carlisle, “Flowgorithm: Graphical authoring tool using flowcharts,” <http://www.flowgorithm.org/>, 2011.
- [5] Replit, Inc., “Replit: The collaborative browser-based IDE,” <https://replit.com/>, 2016.
- [6] OnlineGDB, “OnlineGDB: Online compiler and debugger,” <https://www.onlinegdb.com/>, 2015.
- [7] P. J. Guo, “Online Python tutor: Embeddable web-based program visualization for CS education,” in *Proceedings of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE)*, 2013, pp. 579–584.
- [8] M. Ben-Ari, “Constructivism in computer science education,” *Journal of Computers in Mathematics and Science Teaching*, vol. 20, no. 1, pp. 45–73, 2001.
- [9] A. Zakai, “Emscripten: An LLVM-to-JavaScript compiler,” in *Proceedings of the ACM International Conference Companion on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, 2011, pp. 301–312.
- [10] M. Brunsfeld *et al.*, “Tree-sitter: An incremental parsing system for programming tools,” in *Proceedings of the USENIX Workshop on Hot Topics in Operating Systems (HotOS)*, 2018. [Online]. Available: <https://tree-sitter.github.io>
- [11] M. Brunsfeld, “An incremental parsing system for programming tools,” Master’s thesis, Massachusetts Institute of Technology, 2018.
- [12] D. Eastlake, T. Hansen, G. Fowler, L. C. Noll, and K.-P. Vo, “The FNV non-cryptographic hash algorithm,” Internet Engineering Task Force, Internet-Draft, 2019, <https://datatracker.ietf.org/doc/html/draft-eastlake-fnv>.
- [13] C. Böhm and G. Jacopini, “Flow diagrams, turing machines and languages with only two formation rules,” *Communications of the ACM*, vol. 9, no. 5, pp. 366–371, 1966.
- [14] E. W. Dijkstra, “Go to statement considered harmful,” *Communications of the ACM*, vol. 11, no. 3, pp. 147–148, 1968.
- [15] Microsoft Corporation, “Monaco editor,” <https://microsoft.github.io/monaco-editor/>, 2016.
- [16] —, “Monarch: Create declarative language syntax highlighters,” <https://microsoft.github.io/monaco-editor/monarch.html>, 2016.
- [17] B. Lewis, “Debugging backwards in time,” in *Proceedings of the Fifth International Workshop on Automated and Algorithmic Debugging (AADEBUG)*, 2003. [Online]. Available: <https://arxiv.org/abs/cs/0310016>